XG-Boost

XGBoost (Extreme Gradient Boosting) is an advanced implementation of the gradient Boosting Algorithm designed for speed, performance and scalability.

It is a supervised ML algorithm used for
- classification
- Regression
- Ranking problem

Developed by Tianqui Chen, it became widely popular after winning multiple competitions on kaggle.

Boosting : Combining multiple weak learners (trees) to create a strong model.

Gradient : Uses gradient descent to minimize error.

Extreme : optimized for performance & speed.

XGBoost builds trees sequentially where each new tree corrects the errors of the previous one.

Why XGBoost

High accuracy : outperforms RF and Basic boosting

Regularization :- Prevents overfitting using L_1 & L_2 Penalties

Speed & Efficiency :- Parallel processing (within a tree) optimized tree construction

Handles missing values Automatically :- Learns best direction for missing values

Feature importance :- Helps interpret the model

Works Best with : Structured / tabular data

XG-Boost is Both an algorithm and a library. (a software package that implements this algorithm efficiently)

Foundation Loss Function

Loss function measures how wrong your model's prediction are :-

The smaller the loss → the better the model

$$L(y, \hat{y}) = (\hat{y} - y)^2$$

So, model's goal is to find parameters that minimize loss.

Gradient descent the Engine behind learning

Now how do we minimize loss?

We use GD - an optimization algorithm that update model parameters to reduce loss step by step.

We represent our model with parameters (like slope and intercept), compute gradients and update weights

Let w is parameter (slope & intercept)

calculate gradient $\frac{\partial L}{\partial w}$

update weights

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

Here η - learning rate - controls step size

$\frac{\partial L}{\partial w}$ - slope - tells us how to move.

or η controls how big a step we can take and gradient tells us the direction of error.

Gradient points in the direction of maximum increase of loss.

Gradient = slope of loss function

if we move in this direction, loss will increase fastest.

So gradient always points uphill - towards ↑ing loss.

So, we do use negative gradient as we want to minimize the loss.

So - ve gradient tells us how to go downhill

Example gradient = +5

Moving right → loss ↑

So we move left.

$$W_{new} = W_{old} - \eta(5)$$

gradient -5 → moving left → loss ↓

Loss curve = U-shaped - (if the loss is squared difference)

Example We are minimizing a simple quadratic

Loss function

$$L(w) = (w-3)^2$$

This is a parabola with a minimum at $w=3$

because $L=0$

So, our goal is to find $w=3$ using GD.

Step 1 Compute Gradient

$$\frac{\partial L}{\partial w} = 2(w-3)$$

this tells us the slope of the loss curve at any value of w.

Step 2 update Rule

$$w_{new} = w_{old} - \eta \times \frac{\partial L}{\partial w}$$

$$\eta = 0.1$$

$$\frac{\partial L}{\partial w} = 2(w-3)$$

Step	w(old)	gradient (2(w-3))	update w − 0.1 × gd.	wnew
0	0	−6.0	0 − (0.1 × −0.6) = 0.6	0.6
1	0.6	−4.8	0.6 − (0.1 × −4.8) = 1.08	1.08
2	1.08	−3.84	−1.08 − (0.1 × (−3.84) ⇒ 1.46	1.46
3	1.46	−3.08	→ 1.77	1.77

you can see its moving towards 3, the point of minimum loss.

gradients tells direction of steepest ↑ of loss and magnitude of the slope.

if we take

$$w_{new} = w_{old} + \eta \times grad.$$

$$\Rightarrow 0.6 + 0.1(-0.6)$$

$$= 0.54$$

So, grad $= 2(w-3)$

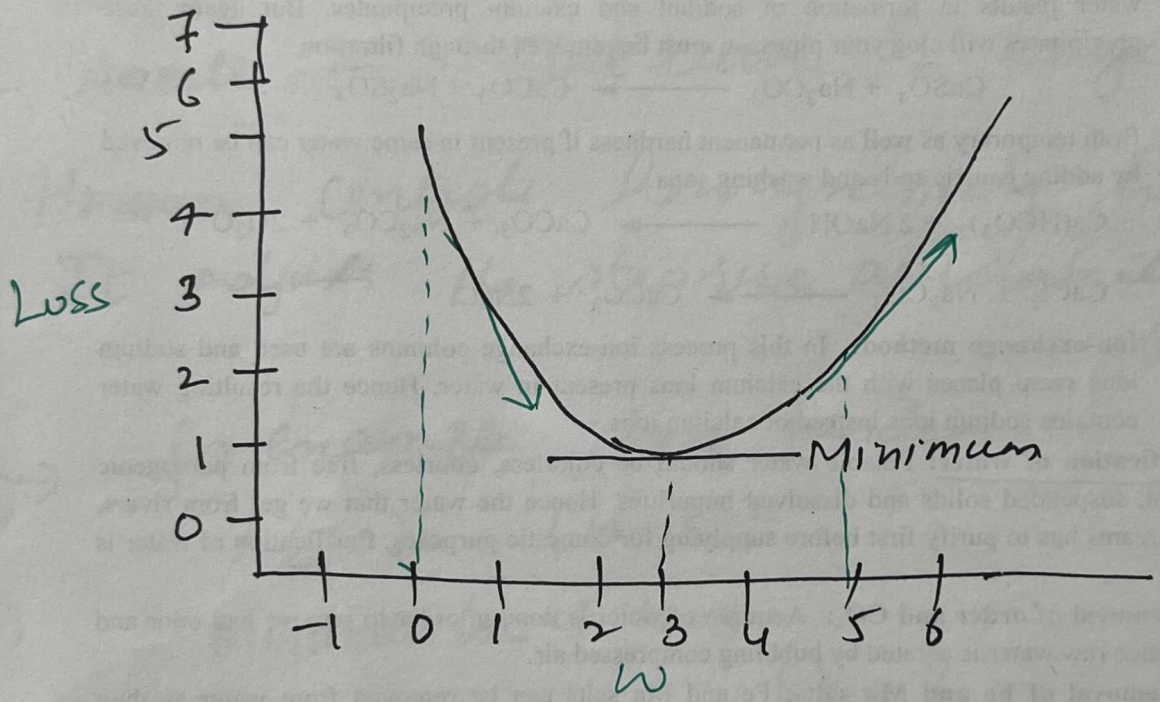
$= 2(0.54 - 3)$

$= -4.92$

$w_{new} = 0.54 + 0.1(-4.92)$

$\Rightarrow 0.048$

So we are moving away from $w=3$ where loss is minimum.



$w=0$, slope is -ve, so moving towards right ($\uparrow w$)

$w=5$, , slope is +ve, so we need to $\downarrow$ weight to get minimum loss

Gradient descent does not know how curved function is, it tells only direction

Here comes newton's method (Hessian) (7)

$$W_{new} = W_{old} - \frac{\frac{\partial L}{\partial W}}{\frac{\partial^2 L}{\partial W^2}}$$

we deride by the second derivative - the Hessian . this gives us smarter updates

Hessian

Large - Steep curve - small step
small - Flat curve - Large step

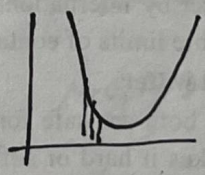
Hessian controls how agressively we update
It adjusts the step size automatically

for example : Loss function

$L(w) = (w-5)^2$

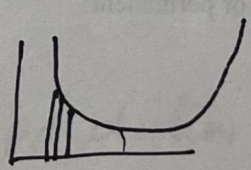
Minimum loss at $w=5$

Steep curve - curve is sharply bending



small change in w → big change in loss
model is very sensitive

Flat curve



Large change in w - small change in loss
model is relaxed

XG-Boost = Gradient Descent + Newton's Method + Trees.

Instead of updating weights like GD, XGBoost updates predictions using trees

Gradient → direction of error

Hessian - confidence / curvature.

Leaf value

$$w = \frac{-\Sigma g_i}{\Sigma h_i + \lambda}$$

Large Hessian → leaf value ↓

So a small update, conservative learning, model is confident - moves carefully - small corrections

small Hessian → leaf value ↑

Large update, Aggressive learning model is uncertain, it makes bigger corrections

Working

(1) Start with initial prediction (mean)
(2) Calculate error (residuals)
(3) Train a tree on errors.
(4) Add corrections
(5) Repeat

So XG Boost is not predicting values directly, it's predicting errors and correcting them step by step.

Iterative model update

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

old prediction plus correction gives new prediction. This correction comes from a tree

XG Boost at each step minimize the following objective Function :

$$\mathcal{L} = \underbrace{\sum_{i=1}^{n} l(y_i, \hat{y}_i)}_{\text{errors (data loss)}} + \underbrace{\sum_{k=1}^{K} \Omega(f_k)}_{\substack{\text{Regularisation} \\ \text{model complexity}}}$$

So, XG Boost is optimizing two things simultaneously fit the data well, but don't overfit
not just trying to fit the data, - also trying to stay simple

Regularization

$$\Omega(f) = \underbrace{\gamma T}_{\downarrow} + \frac{1}{2}\lambda \sum \omega_j^2 \quad \searrow$$

penalizes number of leaves

penalize large leaf values

XG Boost = Gradient + Hessian + Trees + Regularization

Original objective

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

↓ Total loss.

Let's see one iteration (t)

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

↑ adding new tree to improve the prediction

Substitute into loss

So loss becomes $l(y_i, \hat{y}_i^{(t-1)} + \underbrace{f_t(x_i)}_{\text{tree}})$

- each tree splits data
- assign a value to each leaf ⇒ correction

~~Above~~

New loss = Old loss + Linear change + Curvature correction

$$l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx l(\hat{y}_i, \hat{y}_i^{(t-1)} + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2$$

Taylor expression ←

g_i = gradient = $\frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i}$ (direction & error)

if we change prediction slightly, how will loss change.

Hessian h_i = $\frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}$

curvature / confidence

How fast is the gradient changing.

we are converting a complex function into a quadratic approximation using gradient hessian

Drop constant term $l(y_i, \hat{y}^{(t-1)})$

final form

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

Now optimization becomes easy because this is a quadratic function

Regularization

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$$

$\gamma \rightarrow$ penalize number of leaves.

more leaves - more splits - more complex tree

$\gamma \rightarrow$ add a cost for each leaf

Tree A 2 leaves

$\gamma = 1$

Penalty = 2

Tree B 5 leaves

$\gamma = 1$

Penalty = 5

Tree B gets higher penality

(2) λ → penalize leaf values

Leave value - Correction

Large value - aggressive corrections

Case 1 Leaf values [-2, 3]

$\sum w^2 = 4 + 9 = 13$

Case 2 Leaf values [-10, 12]

$\sum w^2 = 100 + 144 = 244$

Case 2 gets much higher penalty

λ prevents the model from making extreme corrections

Leaf value $w = -\frac{\sum g_i \text{ (total error)}}{\sum h_i + \lambda \text{ (control strength)}}$

each leaf value is actually a Newton update - it is deciding the optimal correction using both gradient and hessian

Key - Hyperparameters

n_estimators → no. of trees

max_depth → depth of tree

learning_rate → step size

subsample - row sampling

gamma - minimum loss reduction

colsample_bytree = feature sampling

Example

S-1 Initial prediction $\hat{y}^{(0)} = 20$

x	y	$\hat{y}$
1	10	20
2	20	20
3	30	20

S-2 Compute gradient & Hessian

$$g = 2(\hat{y} - y)$$

$$h = 2$$

x	y	$\hat{y}$	g	h
1	10	20	+20	2
2	20	20	0	2
3	30	20	-20	2

g - tells us direction & Tree

h - tells us how confident we are

S-3 Build first tree

$x < 2.5$

Left node $X(1,2)$

$g = (20, 0)$

Sum = 20

Right node $(x=3)$

Gradient = -20

S-4 Compute leaf values

$$w = \frac{-\sum g}{\sum h + \lambda}$$

Assume $\lambda = 1$

Left leaf $w = \frac{-20}{4+1} = -4$

Right leaf $w = \frac{-(-20)}{2+1} = 6.67$

Leaf value is the correction. -ve means reduce prediction, +ve means increase it

S-5 update prediction

$$\hat{y}^{(1)} = \hat{y}^{(0)} + f_1(x)$$

x	(old) $\hat{y}$	update	New $\hat{y}$
1	20	-4	16
	20	-4	16
2			
3	20	+6.67	26.67

S-6 check error Reduction

	Actual	New $\hat{y}$	Error
x			
1	10	16	6
2	20	16	4
3	30	26.67	3.33

Step 7 Repeat (second tree)

x	y	$\hat{y}$	g
1	10	16	+12
2	20	16	−8
3	30	26.67	−6.67

Since errors are smaller, so correction will also be smaller.

Step 8 New leaf values

Left node $w = \frac{-4}{5} = -0.8$

Right node = $w = \frac{6.67}{3} = 2.22$

updates are getting smaller, this is convergence.

Step 9 update Again

x	y	old $\hat{y}$	update	new $\hat{y}$
1	10	16	−0.8	15.2
2	20	16	−0.8	15.2
3	30	26.67	2.22	28.89